ROLL NO.:24F3003255 NAME: M KATHIRESAN

Trekking Management Application

MAD 2 - Project

Student Details

Name: M. Kathiresan
Email: 24f3003255@ds.study.iitm.ac.in
Program: IITM BS in Data Science and Programming

Project Details

Title of the project

Trekking Management System

Project Overview

The objective of this project is to develop a centralized online platform that simplifies the planning and management of trekking activities. The application enables adventure enthusiasts to discover trekking events, reserve slots, receive important notifications, and monitor their bookings from a single dashboard.

Administrators can efficiently organize trekking schedules, assign staff members, supervise participants, and analyze trekking statistics through an interactive management portal. The system also automates repetitive tasks such as email notifications and report generation to reduce manual effort.

Development Methodology

The project follows a modular web application architecture developed using the Flask framework. Different functionalities are separated into independent modules, making the application easier to maintain and extend.

Database operations are handled through SQLAlchemy ORM, while authentication is implemented using JSON Web Tokens (JWT). Background services execute asynchronously with Celery, allowing long-running operations such as report generation and reminder emails to execute independently of user requests.

Redis is used both as the Celery message broker and as the caching server to improve response time for frequently accessed data.

AI / LLM Declaration

Artificial Intelligence tools were used only as a supporting resource during development. AI assisted in understanding certain framework concepts, debugging errors, and suggesting implementation approaches. To strengthen understanding and implementation skills, various YouTube tutorials were also referenced, offering practical insights into the technologies used.

The complete system design, database schema, backend implementation, API development, testing, integration, and validation were carried out manually.

Estimated AI contribution: **Approximately 20%.**

Technologies and Frameworks Used

Component	**Technology Used**
Programming Language	Python
Backend Framework	Flask
Database	SQLite
ORM	SQLAlchemy
Authentication	Flask-JWT-Extended
Background Processing	Celery
Message Broker	Redis
Caching	Flask-Caching
Email Service	Flask-Mail
Frontend	HTML5, CSS3, Bootstrap 5, JavaScript(vuejs)

Tables

- **User** – Stores the details of all registered users, including Administrators, Staff, and Trek Participants.
- **Staff Profile** – Stores additional information about staff members, such as contact details and their status.

- **Trek** – Stores trekking event information, including location, duration, difficulty level, available slots, assigned staff, schedule, and description.
- **Booking** – Stores user booking records for trekking events along with booking and payment status.
- **Trek Statistics** – Stores summarized information about each trek, including the total number of bookings and completed participants.

Relationships Overview

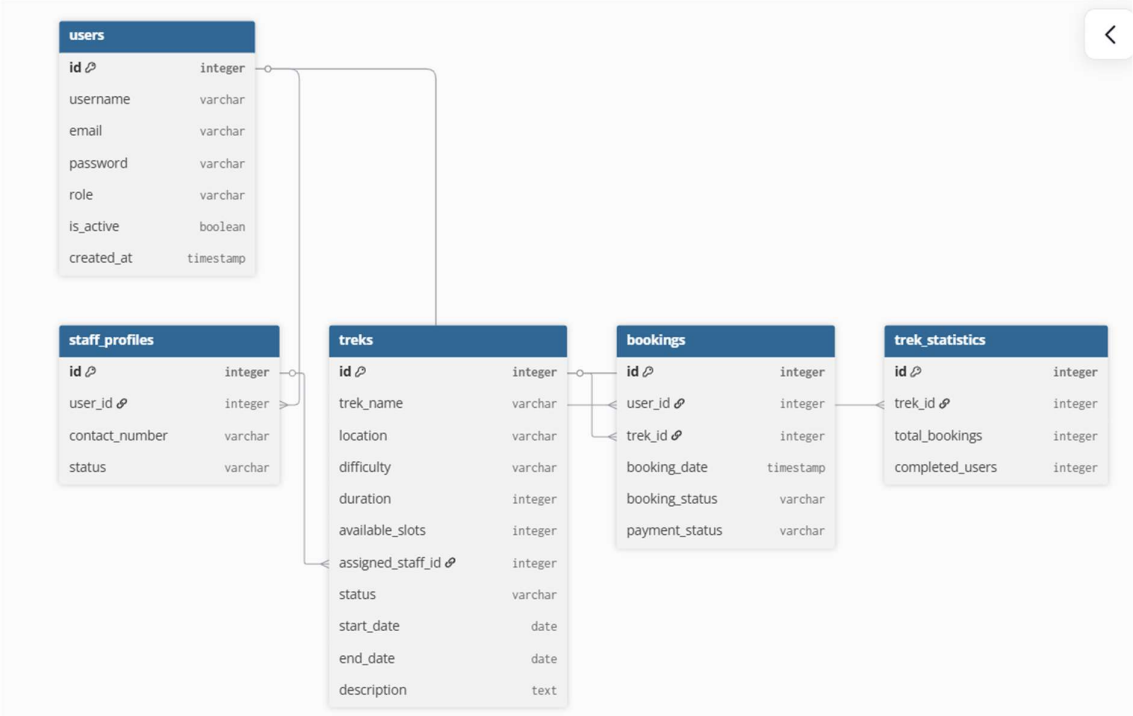
One-to-Many → User → Staff Profile
Each user who has a staff role can have one associated staff profile containing additional staff details.

One-to-Many → Staff Profile → Trek
A single staff member can be assigned to manage multiple trekking events.

One-to-Many → User → Booking
A single user can make multiple trek bookings.

One-to-Many → Trek → Booking
A single trekking event can have multiple user bookings.

One-to-One (Logical) → Trek → Trek Statistics
Each trekking event has one corresponding statistic record that stores booking and completion summaries.



System Architecture

- **app.py** – Application entry point and Flask initialization.
- **auth.py** – Contains API routes for authentication, Admin, Trek Staff, and Trekker operations.
- **config.py** – Stores application configuration and environment settings.
- **models.py** – Defines all SQLAlchemy database models.
- **decorators.py** – Implements role-based access control decorators.
- **extensions.py** – Initializes Flask extensions such as Flask-Mail.
- **cache.py** – Configures Redis caching for the application.
- **celery_config.py** – Configures Celery workers and Celery Beat scheduler.
- **tasks.py** – Implements asynchronous background jobs such as trek reminder emails, monthly report generation, and CSV export.
- **templates/** – Contains HTML templates for Admin, Trek Staff, and Trekker interfaces.
- **static/** – Stores JavaScript files and other static resources.
- **reports/** – Stores generated monthly HTML reports.
- **exports/** – Stores exported CSV files for trekking history.
- **instance/** – Contains the SQLite database and instance-specific files.
- **requirements.txt** – Lists all Python dependencies required for the project.
- **README.md** – Project documentation and setup instructions.

Core Functionalities

User Module

- User registration and secure login
- Browse available trekking events
- Search treks using multiple filters
- Reserve trekking slots
- Cancel bookings when required
- View complete booking history

Staff Module

- View assigned trekking events
- Monitor participant information
- Manage trekking activities

Administrator Module

- Manage trekking events
- Create and assign staff
- Monitor all bookings
- Manage users
- View system statistics
- Track completed and upcoming treks

Advanced Features

The application includes several automation features that improve efficiency.

- Automatic reminder emails before trekking dates.
- Monthly administrative reports generated in the background.
- Export of booking history in CSV format.
- Redis-based caching for improved application performance.
- Scheduled background jobs using Celery Beat.
- Search and filtering options for users, staff and trekking events.
- Secure REST APIs protected through JWT authentication.
- Database integrity maintained using relational constraints.

7. Project Demonstration Video

Drive Link:
https://drive.google.com/file/d/1RTrfq_ImlcxkMc1qUbFfrB7JZQGVlKvU/view?usp=sharing